

Java OOP Mastery - Phase 2

Encapsulation, Access Control, Getters, Setters, this, and Business Logic

This is a complete study reference for the Phase 2 material covered in our Java OOP journey. The goal is understanding, not memorizing definitions.

1. What Encapsulation Really Means

A weak definition is: "Encapsulation means wrapping data and methods together." That is related, but the useful engineering meaning is deeper.

Encapsulation means keeping an object's internal state protected and exposing controlled ways to work with that state. The class contains the state and the behavior that manages that state, while the outside world interacts through a public API.

```
class BankAccount {  
    private double balance;  
  
    public void deposit(double amount) {  
        if (amount > 0) {  
            balance += amount;  
        }  
    }  
  
    public void withdraw(double amount) {  
        if (amount > 0 && amount <= balance) {  
            balance -= amount;  
        }  
    }  
  
    public double getBalance() {  
        return balance;  
    }  
}
```

What is wrapped here?

```
BankAccount  
|  
+-- DATA / STATE  
|    +-- balance  
|  
+-- BEHAVIOR  
    +-- deposit()  
    +-- withdraw()  
    +-- getBalance()
```

The important point is not merely that data and methods are inside one class. The important point is that `balance` is internal state and the class controls how that state can be read or changed.

2. Why Public Fields Are Weak

```
class BankAccount {  
    public double balance;  
}
```

Outside code can directly manipulate the field:

```
BankAccount account = new BankAccount();
account.balance = -5000;
account.balance = 999999999;
```

The object has no opportunity to enforce its own rules. A bank balance normally should not be freely replaced with arbitrary values.

A stronger design keeps the state private and exposes meaningful operations such as `deposit()` and `withdraw()`.

3. private - The Main Access Boundary

```
class Student {
    private String name;

    void printName() {
        System.out.println(name); // OK: inside Student
    }
}

Student s = new Student();
System.out.println(s.name); // ERROR: name is private
```

`private` means the member is directly accessible only from within its declaring class. This is the strongest everyday tool for protecting internal state.

Mental model: `private` = "this is internal implementation; outside code should not directly touch it."

4. Getter - Controlled Read Access

```
class BankAccount {

    private double balance;

    public double getBalance() {
        return balance;
    }
}
```

A getter is a method that provides controlled read access to private state. It does not magically create encapsulation by itself; it is one part of the public API.

```
System.out.println(account.getBalance());
```

Mental model: getter -> controlled READ.

5. Setter - Controlled Write Access

```
public void setBalance(double balance) {
    this.balance = balance;
}
```

A setter provides controlled write access. But a setter is not automatically good business design. If the domain does not make sense as an arbitrary replacement operation, do not add a setter just because a field exists.

Example: Validation in a Setter

```
class BankAccount {

    private double balance;

    public double getBalance() {
        return balance;
    }

    public void setBalance(double balance) {
```

```

        if (balance < 0) {
            throw new IllegalArgumentException(
                "Balance cannot be negative"
            );
        }

        this.balance = balance;
    }
}

```

Now the class can reject invalid state instead of allowing any value.

6. Getter/Setter Is Not the Whole Idea

A common mistake is to believe that private fields plus getters and setters automatically mean "good encapsulation." They do not. Good encapsulation is about controlled state + meaningful operations + business rules + hiding unnecessary implementation details.

Business-focused BankAccount

```

class BankAccount {

    private double balance;

    public double getBalance() {
        return balance;
    }

    public void deposit(double amount) {

        if (amount <= 0) {
            throw new IllegalArgumentException(
                "Amount must be positive"
            );
        }

        balance += amount;
    }

    public void withdraw(double amount) {

        if (amount <= 0) {
            throw new IllegalArgumentException(
                "Amount must be positive"
            );
        }

        if (amount > balance) {
            throw new IllegalArgumentException(
                "Insufficient balance"
            );
        }

        balance -= amount;
    }
}

```

Notice that there is no `setBalance()`. The domain has meaningful operations: `deposit` and `withdraw`. Those operations can enforce the rules of the object.

Design principle: expose behavior, not necessarily raw data.

7. Why `this.name = name` Exists

```

class Student {

    private String name;

    Student(String name) {

```

```

        this.name = name;
    }
}

```

There are two different things named name:

```

this.name  -> instance field belonging to the current Student object
name      -> constructor parameter

```

this.name = name; means: put the parameter value into the current object's instance field.

Why name = name Is Wrong Here

```

class Student {

    private String name;

    public void setName(String name) {
        name = name;
    }
}

```

When a parameter has the same name as the instance field, the local parameter shadows the field. Both sides of name = name refer to the parameter, so the object's field is not updated.

When this Is Not Required

```

class Student {

    private String name;

    void setStudentName(String newName) {
        name = newName;
    }
}

```

Here there is no naming ambiguity: name is the field and newName is the parameter. Java can resolve the field without this.

8. Access Modifiers

Java has four access levels: private, default/package-private, protected, and public.

Modifier	Core access idea
private	Same class only
default (package-private)	Same package
protected	Same package + subclasses, with Java's inheritance-specific rules
public	Broadly accessible from anywhere allowed by the surrounding class/module rules

Mental hierarchy: private -> smallest access; default -> package; protected -> package + inheritance relationship; public -> widest access.

The exact cross-package protected rule is easiest to understand together with inheritance, so it will be revisited there.

9. Encapsulation and Business Rules

The strongest mental model is: the object should be responsible for maintaining its own valid state.

```

Outside code
|
v

```

```

public method
  |
  v
private state
  |
  v
validation / business rules
  |
  v
controlled state change

```

This lets the implementation change without forcing every caller to understand the internals. For example, a future salary validation rule can be changed inside `setSalary()` without changing all calling code.

10. Not Every Field Needs a Setter

```

class User {

    private final int id;
    private String name;

    public User(int id, String name) {
        this.id = id;
        this.name = name;
    }

    public int getId() {
        return id;
    }

    public String getName() {
        return name;
    }

    public void setName(String name) {
        this.name = name;
    }
}

```

Here the ID can be read but not changed after construction. The name can change because that is a plausible business operation. This is why getters and setters should be designed field by field, based on meaning.

11. Complete Interview-Ready Encapsulation Example

```

class BankAccount {

    private final int accountNumber;
    private String accountHolder;
    private double balance;

    // Constructor
    public BankAccount(int accountNumber, String accountHolder, double balance) {

        if (accountNumber <= 0) {
            throw new IllegalArgumentException("Invalid account number");
        }

        if (accountHolder == null || accountHolder.isBlank()) {
            throw new IllegalArgumentException("Invalid account holder name");
        }

        if (balance < 0) {
            throw new IllegalArgumentException("Balance cannot be negative");
        }

        this.accountNumber = accountNumber;
        this.accountHolder = accountHolder;
    }
}

```

```

        this.balance = balance;
    }

    // Getter: read-only account number
    public int getAccountNumber() {
        return accountNumber;
    }

    public String getAccountHolder() {
        return accountHolder;
    }

    // Setter with validation
    public void setAccountHolder(String accountHolder) {

        if (accountHolder == null || accountHolder.isBlank()) {
            throw new IllegalArgumentException("Invalid account holder name");
        }

        this.accountHolder = accountHolder;
    }

    public double getBalance() {
        return balance;
    }

    // Business behavior
    public void deposit(double amount) {

        if (amount <= 0) {
            throw new IllegalArgumentException(
                "Deposit amount must be greater than zero"
            );
        }

        balance += amount;
    }

    // Business behavior
    public void withdraw(double amount) {

        if (amount <= 0) {
            throw new IllegalArgumentException(
                "Withdrawal amount must be greater than zero"
            );
        }

        if (amount > balance) {
            throw new IllegalArgumentException(
                "Insufficient balance"
            );
        }

        balance -= amount;
    }
}

```

Usage

```

public class Main {

    public static void main(String[] args) {

        BankAccount account =
            new BankAccount(101, "Rahul", 5000);

        System.out.println(account.getBalance());

        account.deposit(2000);
        account.withdraw(1000);
        account.setAccountHolder("Arun");

        System.out.println(account.getAccountNumber());
    }
}

```

```

        System.out.println(account.getAccountHolder());
        System.out.println(account.getBalance());
    }
}

```

12. Encapsulation Checklist for Interviews

- Private state: internal fields are not directly exposed.
- Controlled access: use getters only when reading is appropriate.
- Meaningful behavior: prefer deposit/withdraw/changePassword over arbitrary field replacement when the domain demands it.
- Validation: methods can prevent invalid states.
- Hidden implementation: callers should not need to know internal rules.
- No unnecessary setters: do not create setters automatically for every field.
- final when appropriate: an identifier or other invariant may be assigned once and then only read.

13. Phase 2 Mental Model

```

ENCAPSULATION
|
+-- private state
|
+-- controlled public API
|   |
|   +-- getters when needed
|   +-- setters when appropriate
|   +-- business methods
|
+-- validation / invariants
|
+-- hidden implementation details
|
+-- this.field when a parameter/local shadows the field

```

14. Final Phase 2 Takeaways

Encapsulation is not just "private + getter + setter." The deeper goal is to make the object responsible for controlling its own state.

`this.name = name;` means current-object field = parameter.

`this.name` is an instance field of the current object; it is not a field of the constructor or method itself.

`private` creates an access boundary. Public methods form the API through which outside code interacts with the object.

Business methods are often better than raw setters when the domain contains meaningful operations such as `deposit()`, `withdraw()`, `transfer()`, `verifyPassword()`, or `changePassword()`.

No setter by default. Decide access based on whether the state should actually be replaceable.